COMP0005 – ALGORITHMS

Week 9 – 18 March 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Preview (this week & next)
- Muddiest Point Feedback
- Exercises on Graphs (part 2)

PREVIEW – THIS WEEK

- Topic 5 – Strings
  - 5.1 String sorting
  - 5.2 String searching
- Self-practice
- Lab session: TA Lab – Week 9

PREVIEW – NEXT WEEK

- Topic 5 – Strings
  - 5.3 Substring search
  - 5.4 Compression
- Self-practice
- TA Lab – Week 10
- *No Muddiest Point (Weeks 9 & 10 - Strings)*

Group Coursework Deadline: Monday 24 March @ 4pm

OUTLINE

- Preview (this week & next)
- Muddiest Point Feedback
- Exercises on Graphs (part 2)

TOPIC 4 – MUDDIEST POINT

Question

djikstra's algorithm and the edge-weighted dag algorithm for finding shortest path. djikstra's algorithms pseudocode and the reason for both of the algorithms' time complexities

Posted by Anonymous at Wednesday, 12 March 2025, 3:00 PM (19 hours 14 mins ago)

OUTLINE

- Preview (this week & next)
- Muddiest Point Feedback
- Exercises on Graphs (part 2)

EXERCISES ON GRAPHS (PART 2)

- Weighted Undirected Graphs
  - Minimum Spanning Tree
    - Kruskal
    - Prim
- Weighted Directed Graphs
  - Shortest Paths
    - Dijkstra
    - DAG
    - Bellman-Ford

Priority Queue of Edges ~ E log E

```
class Edge:

    def __init__(self, v, w, weight):
        self.v = v
        self.w = w
        self.weight = weight
```

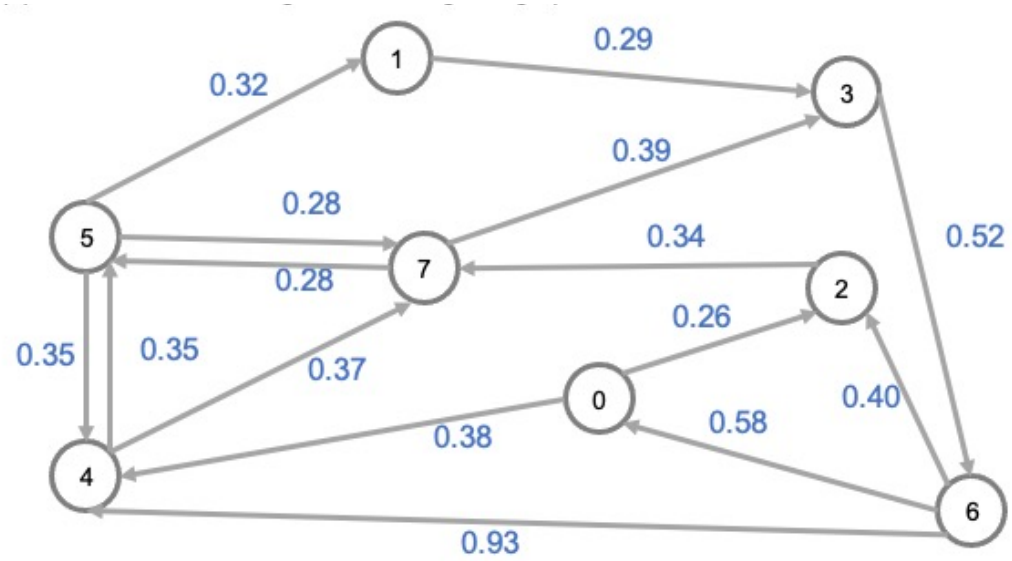
Priority Queue of Vertices ~ E log V

EXERCISES ON GRAPHS (PART 2)

- Weighted Undirected Graphs
  - Minimum Spanning Tree
    - Kruskal
    - Prim
- Weighted Directed Graphs
  - Shortest Paths
    - Dijkstra
    - DAG
    - Bellman-Ford

EXERCISE – SHORTEST PATHS

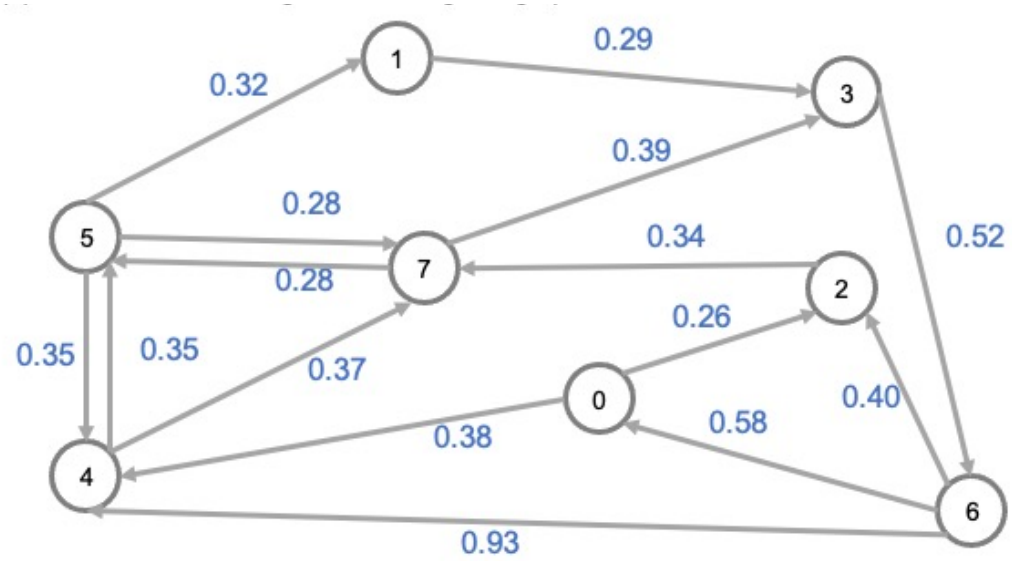
Consider the following directed edge-weighted graph G:



Use the Dijkstra algorithm to compute the shortest path tree rooted in 0. Illustrate the final content of the `distTo[]` vector, keeping track of the minimum distances discovered, and the `edgeTo[]` vector, keeping track of the last edge on the shortest path tree discovered.

EXERCISE – SHORTEST PATHS

Consider the following directed edge-weighted graph G:



Use the Dijkstra algorithm to compute the shortest path tree rooted in 0. Illustrate the final content of the `distTo[]` vector, keeping track of the minimum distances discovered, and the `edgeTo[]` vector, keeping track of the last edge on the shortest path tree discovered.

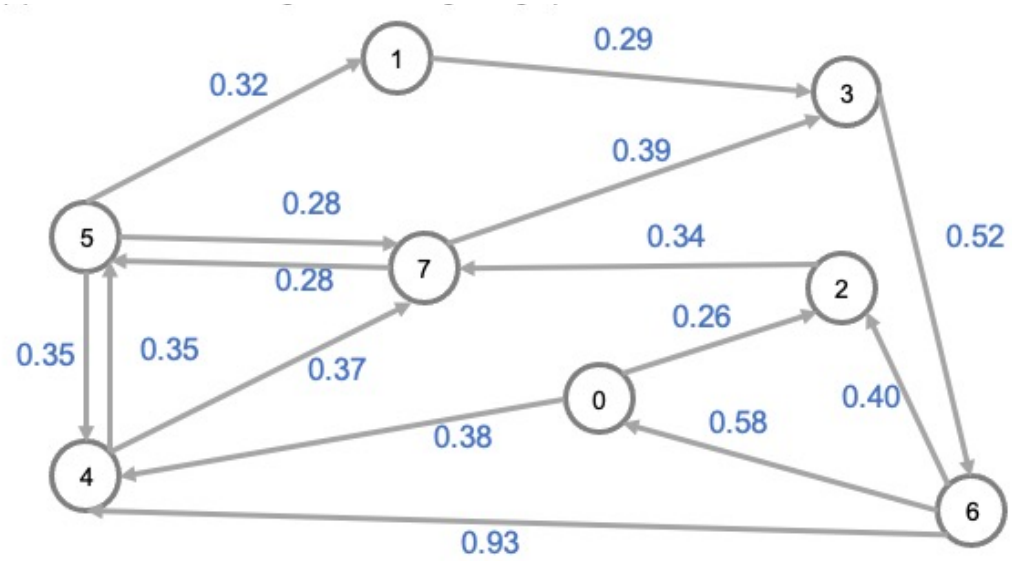
Assumption: non-negative weights
Init: distTo[*s*]=0, distTo[v]=∞

Key idea

- Consider vertex not already in SPT with lowest distTo[] value to *s*
  - **Use a min-oriented PQ**
- Add vertex to the SPT and ***relax*** all its outgoing edges

EXERCISE – SHORTEST PATHS

Consider the following directed edge-weighted graph G:

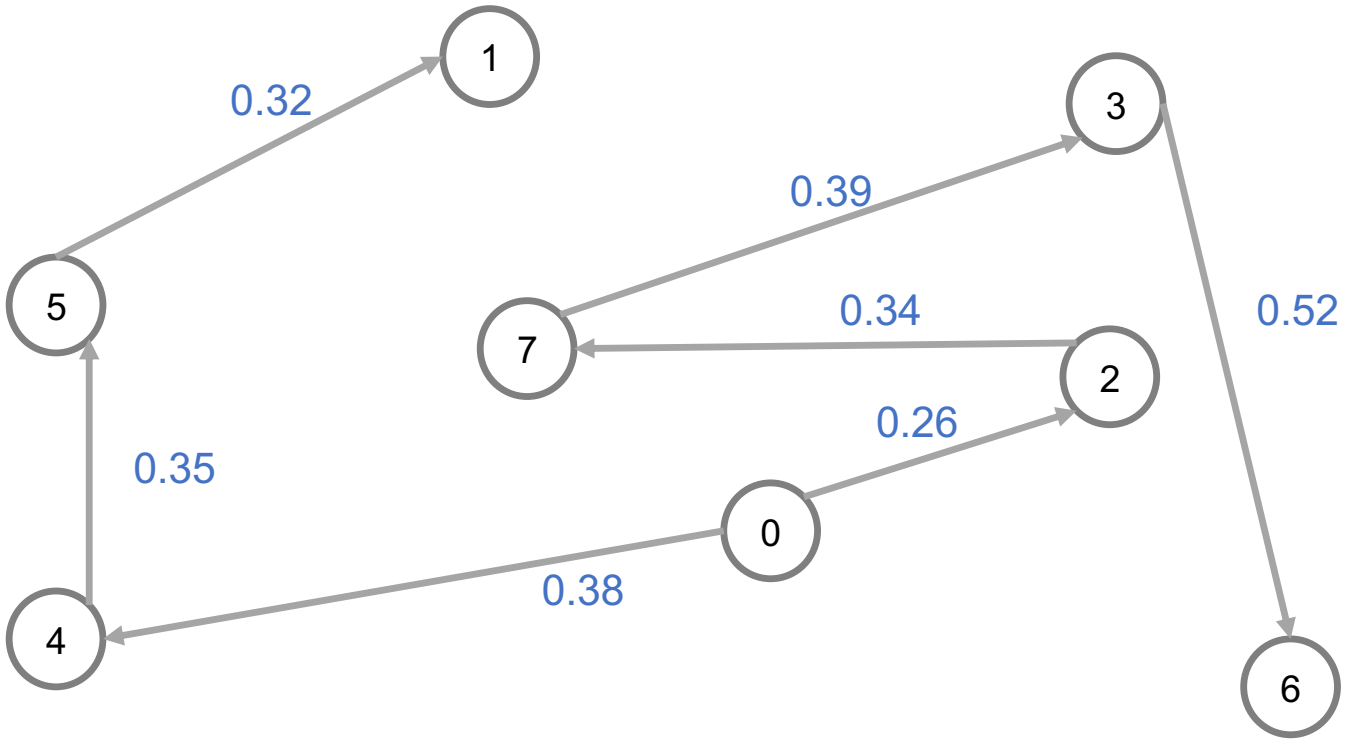


Use the Dijkstra algorithm to compute the shortest path tree rooted in 0. Illustrate the final content of the `distTo[]` vector, keeping track of the minimum distances discovered, and the `edgeTo[]` vector, keeping track of the last edge on the shortest path tree discovered.

```
class DijkstraSP:
   def __init__(G, s):
      edgeTo = [- for v in V]
      distTo = [INF for v in V]
      distTo[s] = 0

      pq = MinPQ()
      pq.insert(s,0)

      while (!pq.isEmpty()):
         v = pq.delMin()
         for e in G.adj(v):
            relax(e)
```

```
   def relax(e):
        u, v = e.fromTo()
        w = e.weight()

        if (distTo[v] > distTo[u] + w):
            distTo[v] = distTo[u] + w
            edgeTo[v] = e

            if (pq.contains(v)):
                pq.decKey(v,w)
            else:
                pq.insert(v, distTo[v])
```

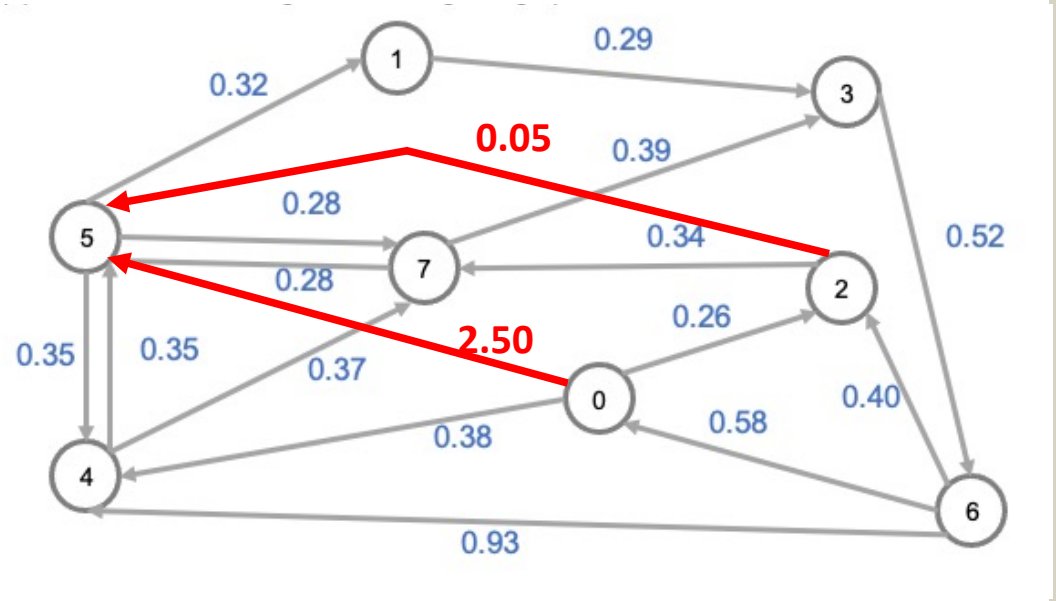
EXERCISE – SHORTEST PATHS



v	distTo	edgeTo
0	0	-
1	1.05	5→1
2	0.26	0→2
3	0.99	7→3
4	0.38	0→4
5	0.73	4→5
6	1.51	3→6
7	0.6	2→7

EXERCISE – SHORTEST PATHS

Consider the following directed edge-weighted graph G:



Use the Dijkstra algorithm to compute the shortest path tree rooted in 0. Illustrate the final content of the `distTo[]` vector, keeping track of the minimum distances discovered, and the `edgeTo[]` vector, keeping track of the last edge on the shortest path tree discovered.

EXERCISE – SHORTEST PATHS

```
class DijkstraSP:
   def __init__(G, s):
      edgeTo = [- for v in V]
      distTo = [INF for v in V]
      distTo[s] = 0

      pq = MinPQ()
      pq.insert(s,0)

      while (!pq.isEmpty()):
         v = pq.delMin()
         for e in G.adj(v):
            u, v = e.fromTo()
            w = e.weight()

            if (distTo[v] > distTo[u] + w):
                  distTo[v] = distTo[u] + w
                  edgeTo[v] = e

                  if (pq.contains(v)):
                        pq.decKey(v,w)
                  else:
                        pq.insert(v, distTo[v])
```

relax(e)

EXERCISE – SHORTEST PATHS

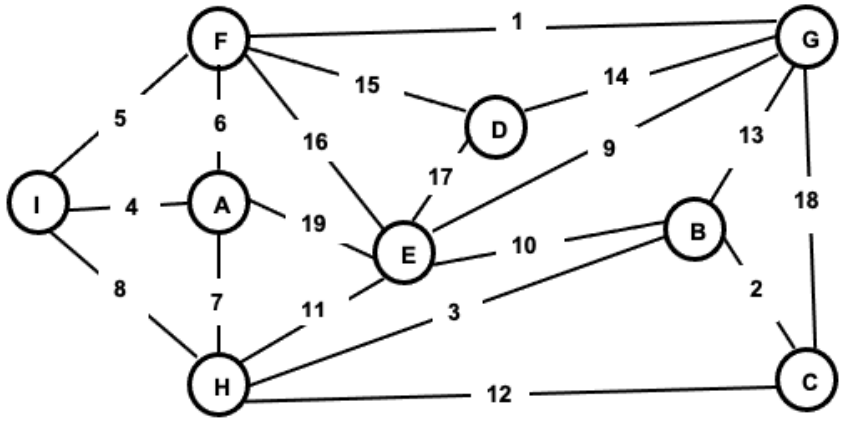
```
class AcyclicSP:
    def __init__(self, G, s):
        self.edgeTo = [None for v in range (0, G.V())]
        self.distTo = [INFINITY for v in range (0, G.V())]
        self.distTo[s] = 0

        dforder = DepthFirstOrder(G)
        topological = dforder.reverse()

        while (!topological.isEmpty()):
            v = topological.pop()
            for e in G.adj(v):
                  v = e.from()
                  w = e.to()
                  if (self.distTo[w] > self.distTo[v] + e.weight()):
                          self.distTo[w] = self.distTo[v] + e.weight()
                          self.edgeTo[w] = e
```

EXERCISES ON GRAPHS (PART 2)

- Weighted Undirected Graphs
  - Minimum Spanning Tree
    - Kruskal
    - Prim
- Weighted Directed Graphs
  - Shortest Paths
    - Dijkstra
    - DAG
    - Bellman-Ford

EXERCISE – MST

Consider the following undirected edge-weighted graph G, comprising 9 vertices and 19 edges:

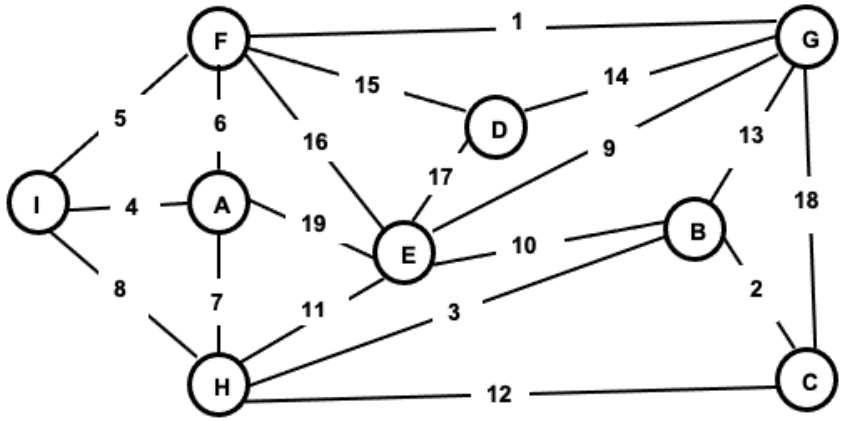


- Briefly describe the main steps performed by Kruskal's algorithm to compute the Minimum Spanning Tree (MST). Use pseudo-code to better illustrate your answer.

- Give the sequence of edges in the MST in the order that Kruskal's algorithm includes them for graph G above.

EXERCISE – MST

Consider the following undirected edge-weighted graph G, comprising 9 vertices and 19 edges:



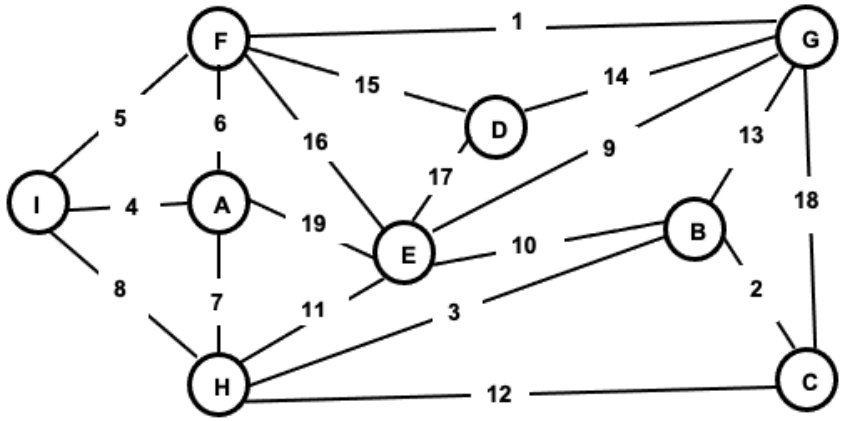
- Briefly describe the main steps performed by Kruskal's algorithm to compute the Minimum Spanning Tree (MST). Use pseudo-code to better illustrate your answer.
- Give the sequence of edges in the MST in the order that Kruskal's algorithm includes them for graph G above.

Main steps

- Consider edges in ascending order of weight
  - Use a **min-oriented priority queue**
- Add the next edge to the MST, unless doing so would create a cycle
  - Use the **Weighted Quick-Union algorithm to detect connected components**
- Stop when all the edges have been considered (or when V-1 edges have been added)

EXERCISE – MST

Consider the following undirected edge-weighted graph G, comprising 9 vertices and 19 edges:



- Briefly describe the main steps performed by Kruskal's algorithm to compute the Minimum Spanning Tree (MST). Use pseudo-code to better illustrate your answer.
- Give the sequence of edges in the MST in the order that Kruskal's algorithm includes them for graph G above.

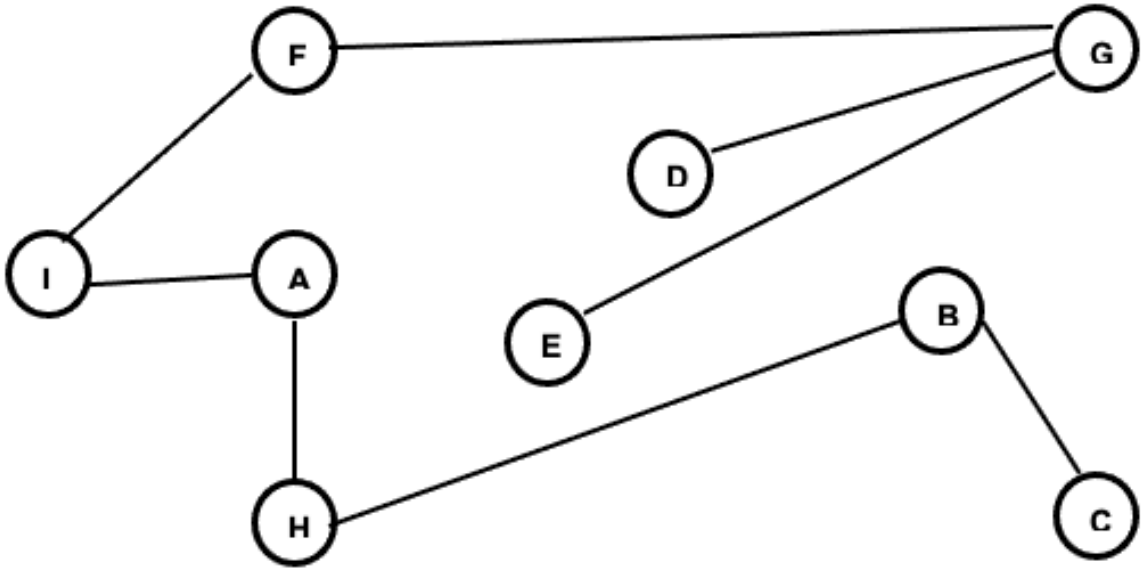
Pseudocode

```
class KruskalMST:
    def __init__(EdgeWeightedGraph G):
        mst = Queue(), pq = MinPQ()
        for e in G.edges()
            pq.insert(e)

        uf = UF(G.V())
        while (!pq.isEmpty() and mst.size()<(G.V()-1)):
            e = pq.delMin()
            v, w = e.endPoints()
            if (!uf.connected(v, w)):
                uf.union(v, w)
                mst.enqueue(e)
```

```
def union(u,v):
    r_u=root(u), r_v=root(v)
    if (r_u != r_v)
        if (size[r_u]<size[r_v]):
            id[r_u]=r_v, size[r_v]+=size[r_u]
        else
            id[r_v]=r_u, size[r_u]+=size[r_v]
```

EXERCISE – MST

Order in which edges are added: FG, BC, BH, AI, FI, AH, EG, DG



EXERCISE – MST

Kruskal – Pseudocode

```
class KruskalMST:
    def __init__(EdgeWeightedGraph G):
        mst = Queue(), pq = MinPQ()
        for e in G.edges()
            pq.insert(e)

        uf = UF(G.V())
        while (!pq.isEmpty() and mst.size()<(G.V()-1)):
            e = pq.delMin()
            v, w = e.endPoints()
            if (!uf.connected(v, w)):
                uf.union(v, w)
                mst.enqueue(e)
```

Prim – Pseudocode

```
class LazyPrimMST:
    def __init__(EdgeWeightedGraph G):
        mst = Queue(), pq = MinPQ()
        marked = [False for v in range (0, G.V())]
        visit(G,0)

        while (!pq.isEmpty() and mst.size() < (G.V()-1)):
            e = pq.delMin()
            v,w = e.endPoints()
            if (marked[v] and marked[w]) skip
            mst.enqueue(e)
            if (!marked[v]) visit(G, v)
            if (!marked[w]) visit(G, w)
```

```
…

    def visit(G, v):
        marked[v] = true
        for (e in G.adj(v)):
            if (!marked[e.otherEndPoint(v)]):
                pq.insert(e)
…
```